

اسم المشروع:

مشروع تخرج — القسم / الكلية:

فريق العمل:

-
-
-
-

نبرة عن المشروع (Abstract)

هذا المشروع عبارة عن نظام متكامل لإدارة صوبة زراعية ذكية (Smart Greenhouse). يتكون من جزأين رئيسيين:

1. **الهاردوير (ATmega32):** يقوم بقراءة بيانات البيئة المحيطة بالنبات (درجة الحرارة، نسبة الرطوبة الجوية، رطوبة التربة، والإضاءة) باستخدام حساسات متطورة. ثم يتخذ قرارات تلقائية لحماية النبات.
2. **تطبيق الموبايل (NET MAUI):** يتصل بالميكروكنترولر عبر البلوتوث لعرض القراءات بشكل حي ومباشر. كما يوفر إمكانية التحكم اليدوي في الأجهزة (مضخة، مروحة، إضاءة).

رسم هيكلي للمشروع (Block Diagram)

- **المدخلات (Sensors):** حساس حرارة ورطوبة (DHT11) + حساس رطوبة تربة (Analog) + حساس إضاءة (LDR).
- **المعالجة (Microcontroller):** ATmega32 يعمل بتردد 8MHz.
- **المخرجات (Actuators):** ريلاي لمضخة مياه + ريلاي لمروحة تبريد + لمبة إضاءة LED عبر ترانزستور.
- **الاتصال (Communication):** وحدة بلوتوث HC-05 ترسل وتستقبل البيانات إلى/من تطبيق الموبايل.

التوصيلات الكهربائية وبروتوس (Proteus Wiring)

المكون / الحساس	الطرف في ATmega32	ملاحظات التوصيل
حساس رطوبة التربة	PA0	مخرج Analog
حساس الضوء (LDR)	PA1	مع مقاومة سحب لأسفل 10kΩ
حساس (DHT11)	PA2	مع مقاومة رفع 4.7kΩ
ريلاي المضخة (Pump)	PB0	يعمل بإشارة منخفضة (Active LOW)
ريلاي المروحة (Fan)	PB1	يعمل بإشارة منخفضة (Active LOW)
الإضاءة (LEDs)	PB2	مفتاح ترانزستور (BC547) لحماية المعالج
وحدة البلوتوث (RX)	PD1 (TX)	لإرسال البيانات
وحدة البلوتوث (TX)	PD0 (RX)	لاستقبال الأوامر

أكواد المشروع الأساسية

1. كود الهاردوير (ATmega32 - Logic Layer)

هذا الجزء يوضح محرك القرار (State Machine) وكيفية التعامل مع الحساسات:

```
static void state_evaluate(void) {
    /* Pump Logic (with hysteresis) */
    if (g_pump_mode == MODE_AUTO) {
        if (g_data.soil < SOIL_THRESHOLD_LOW) g_data.pump_on = 1;
        else if (g_data.soil > SOIL_THRESHOLD_HIGH) g_data.pump_on = 0;
    } else {
        g_data.pump_on = (g_pump_mode == MODE_MANUAL_ON) ? 1 : 0;
    }

    /* Fan Logic (with hysteresis) */
    if (g_fan_mode == MODE_AUTO) {
```

```

        if (g_data.temperature > TEMP_THRESHOLD_HIGH) g_data.fan_on = 1;
        else if (g_data.temperature < TEMP_THRESHOLD_LOW) g_data.fan_on = 0;
    } else {
        g_data.fan_on = (g_fan_mode == MODE_MANUAL_ON) ? 1 : 0;
    }
}

```

2. كود تطبيق الموبايل (MAUI - DashboardViewModel)

كيفية معالجة أوامر التحكم اليدوي (Manual Override) وإرسالها عبر البلوتوث:

```

private void TogglePump() {
    string cmd = IsPumpOn ? "P:OFF" : "P:ON";
    _bluetooth.SendCommand(cmd);
}

private void ToggleFan() {
    string cmd = IsFanOn ? "F:OFF" : "F:ON";
    _bluetooth.SendCommand(cmd);
}

private void ResetAuto() {
    _bluetooth.SendCommand("A:AUTO");
    StatusMessage = "🔄 Switched to AUTO mode";
}

```

? أسئلة المناقشة المتوقعة للمشروع

س1: لماذا تم بناء الكود باستخدام `State Machine` بدلاً من `delay`؟

الإجابة: لأن استخدام `delay` يوقف المعالج تمامًا، مما يمنع استقبال أوامر البلوتوث للتحكم اليدوي. الـ State Machine تجعل النظام (Non-blocking) ومستجيب دائمًا للأوامر في أي لحظة.

س2: كيف تم تنفيذ خاصية الـ (Hysteresis) في المروحة والمضخة؟ ولماذا؟

الإجابة: بوضع عتبتين مختلفتين للتشغيل والإيقاف (مثل التشغيل عند 30°C والإيقاف عند 27°C). السبب هو منع الريلاي من الارتعاش (Chattering) وتخريب الأجهزة عند تذبذب الحرارة حول درجة معينة.

س3: ما هي بنية `MVVM` المستخدمة في تطبيق الموبايل؟

الإجابة: هي (Model - View - ViewModel)، وهي تفصل واجهة المستخدم عن منطق التشغيل. هذا يسمح بتحديث الشاشة تلقائياً بمجرد وصول بيانات من البلوتوث باستخدام خاصية (Data Binding) دون كتابة كود معقد في الواجهة.

س4: كيف تمكّن التطبيق من قراءة بيانات البلوتوث دون أن يتجمد (Freezing)؟

الإجابة: قراءة البلوتوث تتم في مسار خلفي منفصل (`Task.Run`). عندما تصل البيانات، يتم تمريرها للواجهة الرئيسية عبر دالة `MainThread.BeginInvokeOnMainThread` لتحديث الـ UI بأمان تام.